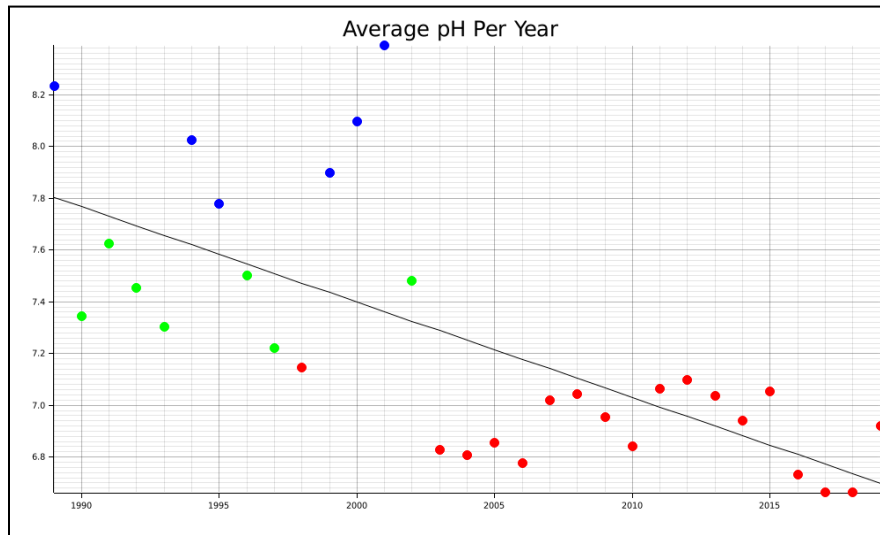


How has water quality changed over time ?
Elvin H.



Data from: <https://catalog.data.gov/dataset/water-quality-data-41c5e>

A. Project Overview

Throughout my time in classes, I've always been drawn to the idea of diving into data analysis within the medical field. One of the most fascinating datasets I found was one that tracked pH levels in water over a span of 35-years. pH, which stands for potential of hydrogen, is a scale that tells us how acidic or basic a substance is. Maintaining the right pH and ensuring clean water are crucial for our health. Unfortunately, over the years, the quality of our water has taken a hit due to climate change, industrial pollution, poor waste management, and the growing issue of microplastics. This project allowed me to dig into how pH levels in our water sources have shifted over time, utilizing real environmental data, statistical analysis, and clustering techniques to uncover some intriguing patterns.

B. Data Processing

- *To work with this dataset, I used CSV and Serde crates. I first refined a `WaterQualityRecord` struct using the `#[derive(Deserialize)]` to help automatically map each CSV row to a Rust structure. The `CSV: ReaderBuilder` helped to read the file line by line, and `Serde` helped to parse fields such as pH, temp, and year.*
- *Some of the cleaning I had to do for the data was deleting some columns that weren't necessary, and the main `ph_standard_units` data needed some cleaning. Some of the rows had `None`, so we deleted those rows and then mapped the valid pH readings by year using `HashMap<i32, Vec<f64>>`, which helped a lot for computing averages.*

C. Code Structure

reader.rs (Module): Throughout this module, the main purpose is to handle the reading and parsing of the CSV data into a usable Rust structure. Just like stated above in the data processing, this module handles water quality records as a struct, which helps to represent each row in the CSV dataset. The purpose is to group valid pH values by year, and then we end up with a hashmap where it gets sorted by year and value, where value is the pH readings.

compute_and_plot.rs (Module): This module is to do all the computation and plotting for the project. The first of the main functions that come into play is

- `compute_averages(year_to_ph: &HashMap<i32, Vec<f64>>) -> Vec<(i32, f64)>` in which its purpose is to compute the average pH per year; What it takes in is the maps of years and makes them into pH values; the output helps us sort vectors of years and values (pH values). We do this using loops throughout every year and compute the average using `.iter().sum()`.
- Second is `linear_regression(points: &[(i32, f64)]) -> (f64, f64)`, which fits a line to the data (least squares). It takes the vectors from above (year and pH) and outputs a slope and intercept of the regression line. This uses the summation formulation to compute these.
- Third is `plot_to_png(averages: &[(i32, f64)], clusters: &[usize]) -> Result<(), Box<dyn Error>>` The purpose of this is to plot the average pH values and the colors by cluster: red being low pH levels, blue being high pH, and green being moderate. This then takes in the yearly average pH and cluster labels and helps to output the PNG file for us. Throughout this process, we use plotters to render the points, color them, and draw the regression line that we got above.

cluster.rs (Module): This module helps to make the clustering.

- The `cluster_years` function helps to group years based on the similarity of their average pH values using KMeans clustering. It takes in a vector of (year, average_pH) pairs along with the desired number of clusters.
- The function extracts only the pH values and converts them into a numerical array format (`ndarray::Array2`) required by the linfa machine learning library. Using the KMeans algorithm, it fits a clustering model to this data and returns a vector of labels that assign each year to a cluster based on its pH similarity. The output is used to color-code the years in the final visualization, helping to reveal patterns in water quality.

main.rs (Module): The main module then helps us to put past modules and functions together and make them all function together to run.

- *The main block below helps us connect everything and make everything into simple steps.*
- *let year_to_ph = load_and_group_by_year(file_path)?;*
 - *Helps us read the data with the reader*
- *let averages = compute_averages(&year_to_ph);*
 - *Computes the pH averages with the compute_and_plot module.*
- *let valid_averages = averages.filter(|year| year > 1950 && year < 2100);*
 - *Computes average*
- *let clusters = cluster_years(&valid_averages, 3);*
 - *We then cluster the years using cluster*
- *plot_to_png(&valid_averages, &clusters)?*
 - *Gives us our output, which is a PNG graph with the pH trends and the clusters.*

D. Tests: *My tests are located at the bottom of the main module.*

Test: Our test is `fn test_pipeline_flow()`. Testing every single module/technique used.

- We first off with a mock input data, which is represented as `mock_data.insert(2000, vec![7.0, 7.5, 6.5])`.
- `mock_data.insert(2001, vec![8.0, 6.0]) ;`
- We then compute the average pH for each year.
- Then we filter out invalid years, like how I did above, to keep the test aligned.
- Let clusters then cluster the average pH values into 2 groups using a cluster. rs, making sure it doesn't return invalid results
- Then let `plot_result` call to generate the PNG file, and then assert that no error occurs, aka the plot logic runs correctly.

E. Results:

Unsurprisingly, the results of our data show a downward trend in pH levels over time, as seen in the PNG generated by the code, showing us how badly our society has done to keep our waters safe. The regression line that is included provides a more insightful view of the downward trend, which is crucial for focusing on finding a solution to this problem.

F. Usage Instructions:

The project doesn't require any human interaction; it just uses basic cargo build and cargo run—release to run to analyze our data. It operates on a hardcoded path because we already have the file, as in `let file_path = "Cleaned_WaterQualityData.csv";` this is the reason it looks like it does.

G. AI-Assistance Disclosure and Other Citations

This project was created without the use of an AI assistant or documents, but I did use ChatGPT to learn how to move the project from Red Hat to GitHub. I created my entire project in Red Hat during this process, but I later realized that I needed to move it to GitHub and create commits for the process. As a result, I had to create commits based on the steps I took over time, like creating the reader first, then the computation, and finally the remaining components, like the main.